

Storage–Retrieval–Performance Trade-offs

Designing Efficient Storage and I/O Systems

Prashant Jadav

DEPSTAR - CHARUSAT

Why Storage–Retrieval Trade-offs Matter

- Applications continuously store and retrieve data.
- Storage systems must balance several competing requirements:
 - **Capacity** – how much data can be stored?
 - **Latency** – how quickly can data be retrieved?
 - **Throughput** – how much data can be transferred per unit time?
 - **Cost** – how expensive is the storage?
 - **Reliability** – how safely is data retained?
 - **Energy** – how much power does the system consume?

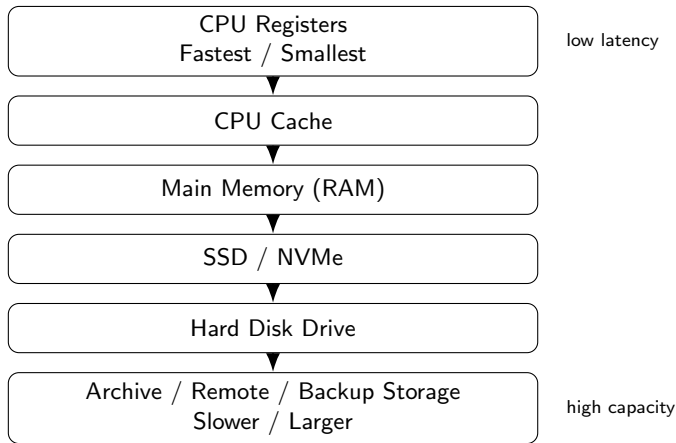
Central idea

There is usually no single storage technology or configuration that simultaneously maximizes capacity, speed, reliability, and low cost.

Example

Keeping all data in high-speed memory provides excellent retrieval performance but is much more expensive per byte than large-capacity secondary storage.

Storage Hierarchy



Principle

Frequently accessed data is kept closer to the CPU, while less frequently accessed data can be placed on larger and slower storage.

Key Storage Performance Metrics

Latency Time between issuing a request and obtaining the requested result.

Throughput Amount of data transferred per unit time.

IOPS Input/output operations completed per second.

Bandwidth Data transfer rate, often expressed in MB/s or GB/s.

Queueing delay Time spent waiting for the storage device to service the request.

Tail latency High-percentile latency such as P_{95} or P_{99} .

Important distinction

A device can have high bandwidth but still have poor latency for small random requests.

Latency vs. Throughput

Latency

- Important for interactive applications.
- Measures how long an individual operation takes.
- Lower latency improves responsiveness.

Throughput

- Important for bulk data processing.
- Measures total work completed over time.
- Higher throughput improves large transfers.

Example

A database lookup may prioritize low latency, while a backup system may prioritize high sequential throughput.

Trade-off

Optimizing one metric does not automatically optimize the other.

Sequential vs. Random Retrieval

Sequential access

- Data is accessed in nearby logical locations.
- Good locality.
- Often supports high throughput.
- Read-ahead can be effective.

Random access

- Requests are distributed across storage locations.
- Less predictable locality.
- Can increase latency on HDDs.
- Requires efficient request scheduling.

OS implication

The same storage device can behave very differently depending on the application's access pattern.

HDD: Storage Capacity vs. Retrieval Performance

- HDDs provide large capacity at relatively low cost per byte.
- Mechanical movement introduces:
 - Seek time
 - Rotational latency
 - Mechanical positioning overhead
- Sequential access can make better use of HDD bandwidth.
- Random access can suffer from repeated head movement.

Design trade-off

HDDs are attractive for high-capacity storage, but applications requiring consistently low random-access latency may benefit from faster storage technologies.

SSD: Low Latency with Different Trade-offs

- SSDs use flash memory rather than mechanical platters and heads.
- No mechanical seek or rotational latency.
- Strong random-access performance compared with HDDs.
- High internal parallelism can support substantial I/O concurrency.

New trade-offs

SSD systems still involve considerations such as write endurance, garbage collection, write amplification, thermal behavior, capacity, and cost.

Key point

Removing mechanical latency changes the performance bottleneck; it does not eliminate the need for storage-system optimization.

RAM vs. Secondary Storage

Property	RAM	SSD / HDD
Latency	Very low	Higher
Capacity	Smaller	Much larger
Volatility	Volatile	Non-volatile
Cost per byte	Higher	Lower
Persistence	Lost when power is removed	Retained
Typical role	Active working data	Persistent storage

Why use both?

RAM provides fast retrieval for active data, while secondary storage provides persistence and much larger capacity.

Caching: Trading Memory for Retrieval Speed

- A cache stores frequently or recently used data in a faster layer.
- A cache hit avoids accessing the slower storage layer.
- Cache effectiveness depends on locality and workload characteristics.

$$T_{\text{avg}} = hT_{\text{fast}} + (1 - h)T_{\text{slow}}$$

where:

- h = cache hit ratio
- T_{fast} = fast-layer access time
- T_{slow} = slow-layer access time

Trade-off

More cache capacity can improve the hit ratio, but consumes valuable memory.

Numerical Example: Cache Hit Ratio

Suppose:

$$T_{\text{cache}} = 1 \text{ ns}$$

$$T_{\text{storage}} = 10 \text{ ms}$$

and the cache hit ratio is:

$$h = 0.95$$

Then:

$$T_{\text{avg}} = 0.95(1 \text{ ns}) + 0.05(10 \text{ ms})$$

Since:

$$10 \text{ ms} = 10,000,000 \text{ ns}$$

$$T_{\text{avg}} \approx 0.95 + 500,000 = 500,000.95 \text{ ns}$$

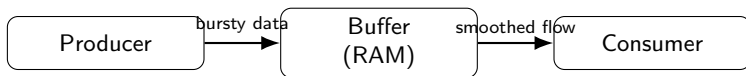
$$T_{\text{avg}} \approx 0.5 \text{ ms}$$

Observation

Even a high hit ratio may not completely eliminate the impact of a very slow storage miss.

Buffering: Trading Memory for Smooth I/O

- A buffer temporarily stores data while producer and consumer operate at different rates.
- Useful when:
 - Device and CPU speeds differ.
 - Data arrives in bursts.
 - Large sequential transfers are possible.

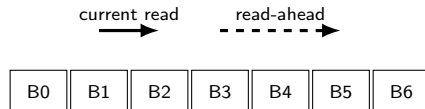


Trade-off

Larger buffers can absorb larger bursts, but consume more memory and may increase delay if data is allowed to accumulate excessively.

Read-Ahead: Predicting Future Retrieval

- The OS can predict that nearby blocks will soon be requested.
- It retrieves additional blocks before the application explicitly asks for them.
- Particularly useful for sequential file access.



Risk

If the prediction is wrong, unnecessary data is fetched and memory bandwidth is wasted.

Write-Back: Trading Durability for Performance

Write-through

- Update storage immediately.
- Better persistence visibility.
- More storage writes.

Write-back

- Modify cached data first.
- Write to storage later.
- Can combine multiple writes.
- Requires careful recovery.

Trade-off

Write-back can improve performance by reducing and combining physical writes, but important data may temporarily exist only in volatile memory.

I/O Scheduling and Retrieval Performance

- Multiple requests may compete for the same storage device.
- The OS can reorder requests to improve performance.
- Classical HDD scheduling includes:
 - FCFS
 - SSTF
 - SCAN
 - C-SCAN
 - LOOK

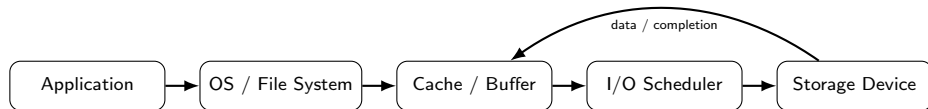
Trade-off

Reducing physical movement may improve average latency, but an aggressive optimization can affect fairness or tail latency.

Mixed workload

A background sequential backup and latency-sensitive random database reads may require different scheduling treatment.

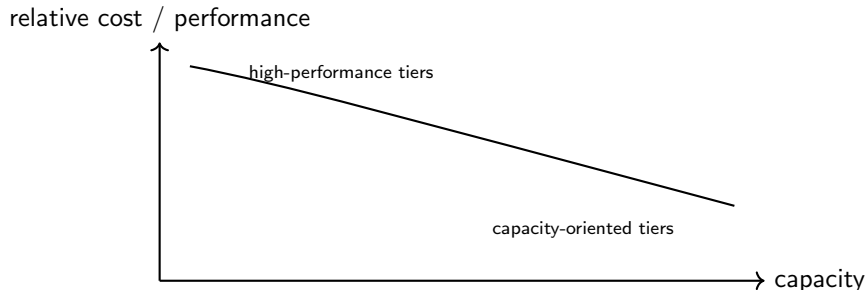
Storage Performance: A Layered View



- Each layer can affect end-to-end retrieval performance.
- A fast device cannot compensate for inefficient software indefinitely.
- Optimization should consider the complete I/O path.

Capacity vs. Performance

- Increasing capacity does not necessarily increase retrieval performance.
- Large-capacity devices are often optimized for cost-effective persistent storage.
- High-performance storage may have higher cost per byte.



Design principle

Choose the storage tier according to workload requirements rather than maximizing one property.

Reliability vs. Performance

- Reliability mechanisms often introduce additional work.
- Examples:
 - Replication
 - Journaling
 - Checksums
 - Synchronous writes
 - Redundant storage
- Stronger durability guarantees can increase write latency or storage overhead.

Example

A synchronous write may wait until data is safely committed to persistent storage, whereas a buffered write may return earlier but provide a weaker immediate durability guarantee.

Key insight

Performance optimization must not silently violate the application's required reliability or durability guarantees.

Cost vs. Performance

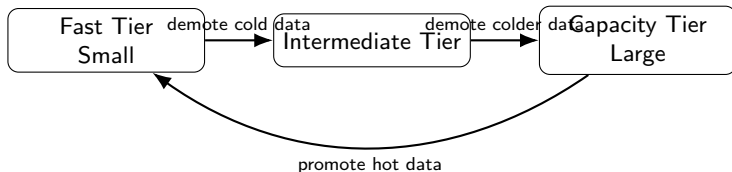
Storage tier	Typical strength	Typical limitation	Suitable use
RAM	Very low latency	Expensive, volatile	Active working set
NVMe SSD	High I/O performance	Higher cost than HDD	Databases, applications
SATA SSD	Good latency	Lower performance than high-end NVMe	General-purpose storage
HDD	High capacity / low cost per byte	Mechanical latency	Bulk storage, archives
Remote / archive	Large capacity	Network/access latency	Backup and long-term storage

Architecture

Many systems use a **storage hierarchy** instead of relying on one storage technology.

Working Set and Storage Placement

- The **working set** is the subset of data actively needed by an application during a period.
- Keeping the working set in a faster tier can greatly improve retrieval performance.
- Less frequently accessed data can remain on slower, larger storage.



Trade-off

Tiering attempts to obtain near-fast-tier performance for frequently accessed data without storing the entire dataset on the most expensive tier.

Queueing Effects

- Storage performance depends not only on device speed but also on queue depth.
- When request arrival rate approaches or exceeds service capacity, queueing delay increases.
- Under heavy load, a fast device can still produce high application latency.

$$T_{\text{response}} = T_{\text{queue}} + T_{\text{service}}$$

Important

Reducing device service time alone may not solve a latency problem if the dominant component is queueing delay.

Performance Example: Queueing

Suppose a storage request experiences:

$$T_{\text{queue}} = 4 \text{ ms}$$

and

$$T_{\text{service}} = 2 \text{ ms}$$

Then:

$$T_{\text{response}} = 4 + 2 = 6 \text{ ms}$$

Now suppose a faster device reduces service time to:

$$T_{\text{service}} = 1 \text{ ms}$$

but queueing remains 4 ms:

$$T_{\text{response}} = 4 + 1 = 5 \text{ ms}$$

Observation

Amdahl's-Law View of Storage Optimization

If only a fraction f of execution time is affected by a storage optimization with speedup S :

$$S_{\text{overall}} = \frac{1}{(1 - f) + \frac{f}{S}}$$

Example

If storage operations account for 30% of execution time and storage is made $4\times$ faster:

$$S_{\text{overall}} = \frac{1}{0.7 + \frac{0.3}{4}} \approx 1.29$$

Lesson

Optimizing storage provides limited overall benefit if storage is only a small part of the application's total execution time.

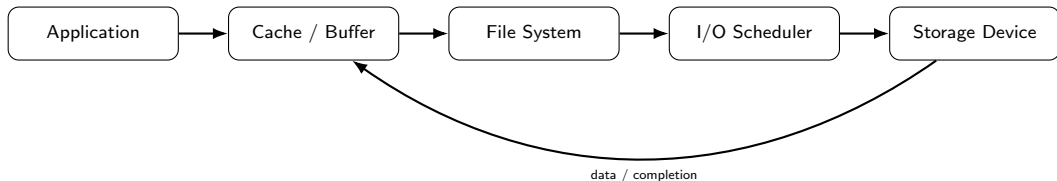
Retrieval Performance: What Should Be Optimized?

- **Small random I/O** → prioritize latency and IOPS.
- **Large sequential I/O** → prioritize bandwidth and efficient transfer.
- **Interactive applications** → prioritize response time and tail latency.
- **Backup/archive** → prioritize capacity, throughput, and cost.
- **Critical databases** → balance latency, durability, consistency, and throughput.

No universal optimum

The appropriate storage design depends on the application's workload and service-level requirements.

Integrated Storage Design



- Caching reduces physical accesses.
- Buffering absorbs rate differences.
- File systems organize persistent data.
- Scheduling determines request order.
- Storage hardware determines the physical service characteristics.

Case Study: Database Server

Workload

A database server has:

- Frequent small random reads
 - Transactional writes
 - Periodic large backups
-
- Database pages benefit from RAM caching.
 - Low read latency is important for interactive queries.
 - Write durability may require controlled persistence.
 - Backups should avoid overwhelming foreground database I/O.

Design trade-off

Maximizing backup throughput at the expense of database tail latency may violate the application's performance requirements.

Case Study: Video Streaming

- Video workloads often perform large sequential reads.
- High sustained bandwidth is more important than extremely low latency for every individual block.
- Read-ahead and buffering can smooth delivery.
- Sufficient buffering can tolerate temporary variations in storage/network throughput.

Trade-off

Larger buffers can reduce playback interruptions caused by temporary throughput variation, but consume more memory and increase startup delay if overused.

Design Checklist

When designing a storage system, ask:

- 1 What is the workload: random or sequential?
- 2 Are reads or writes dominant?
- 3 What latency is acceptable?
- 4 What throughput is required?
- 5 What is the working-set size?
- 6 How much memory can be used for caching?
- 7 What durability guarantees are required?
- 8 Is the device an HDD, SSD, or remote storage system?
- 9 What is the cost per byte?
- 10 What happens under peak load?

Higher-Order Design Problem

Scenario

A university data server stores 50 TB of data. It serves interactive student applications, database queries, and nightly backups.

- 1 Which data should be kept in a fast storage tier?
- 2 How would caching improve retrieval performance?
- 3 Should backup writes receive the same priority as interactive reads?
- 4 When would read-ahead be beneficial?
- 5 How would you balance capacity and retrieval latency?
- 6 Which metrics should be monitored: average latency, P_{99} latency, throughput, IOPS, or all of them?
- 7 Explain how the design should change if the primary storage changes from HDD to SSD.

Key Takeaways

- Storage design is a trade-off among **latency, throughput, capacity, cost, reliability, and energy**.
- Storage hierarchies place frequently accessed data in faster tiers.
- Caching trades memory capacity for faster retrieval.
- Buffering trades memory for smoother data transfer.
- Read-ahead improves sequential workloads but can waste resources when predictions are wrong.
- Write-back can improve performance but requires careful durability handling.
- Queueing delay can dominate total response time even when the storage device is fast.
- Storage optimization should be driven by the application's actual workload and service requirements.

The fastest storage is not always the best storage; the right design matches storage characteristics to workload requirements.

Quick Check

- 1 Differentiate latency, bandwidth, and IOPS.
- 2 Why does sequential access generally favor high throughput?
- 3 How does caching improve average retrieval time?
- 4 What is the risk of aggressive read-ahead?
- 5 Why can write-back improve performance?
- 6 Why can queueing dominate storage response time?
- 7 Explain the capacity–cost–performance trade-off.
- 8 Why is tail latency important for interactive applications?